Gmail

Kunal Deshmukh <deshmukhkunal813@gmail.com>

Module 1 – User Management Module | Implementation Guidelines

1 message

springboardmentor1234x <springboardmentor5432x@gmail.com> 7 October 2026 at 20:44
Bcc: deshmukhkunal813@gmail.com

Interns,

As part of the **CreatorIQ: Creator Analytics & Content Performance Dashboard** project, you are now required to work on **Module 1 – User Management Module**. This is the foundational module of the project because the remaining analytics and dashboard features will depend on proper user authentication, user profiles, and access control.

The objective of this module is to build a secure user management system that allows users to register, log in, manage their profiles, and access the platform according to their assigned roles. The project defines four major roles: **Creator, Agency, Marketing Team, and Administrator**. Each role should have appropriate access to the platform based on its responsibilities.

1. User Registration and Login

The first part of the module is to implement user registration and login functionality.

For registration, users should be able to create an account by providing the required information. You should decide and design the necessary fields for registration based on your system requirements. The backend should validate the input before storing the user information in the database.

For login, the system should verify the user's credentials and allow access only when the credentials are valid. Invalid credentials should be handled properly with an appropriate error response.

The authentication flow should be implemented using the authentication technologies specified in the project, particularly **JWT Authentication and OAuth2 Authentication**.

2. Creator Profile Management

The platform is designed to support content creators, so you need to provide functionality for managing creator profiles.

A creator should be able to view and update relevant profile information. The profile should be connected to the authenticated user so that one creator cannot access or modify another creator's private profile information.

Think about the complete flow:

Registration → Login → Authentication → Creator Profile → Update Profile

Make sure the backend validates the authenticated user before allowing profile-related operations.

3. Agency Profile Management

The system also supports agencies that manage or work with creators.

You should implement the required profile management functionality for agency users. The agency profile should be associated with the correct authenticated account, and access should be restricted according to the user's role.

The objective is not just to create a profile page but to understand how different types of users can exist within the same application while having different permissions.

4. Role-Based Access Control

One of the important requirements of Module 1 is **Role-Based Access Control (RBAC)**.

The project specifies the following roles:

- Creator

- Agency
- Marketing Team
- Administrator

Each role should have appropriate permissions. For example, a Creator should not automatically receive Administrator-level access simply because the user is authenticated.

You should design the authorization logic so that the system first verifies:

Who is the user?

and then:

What is the user's role?

and finally:

Is this role allowed to perform this particular action?

This distinction between **authentication** and **authorization** is important for the project.

Authentication answers:
“Who are you?”

Authorization answers:
“What are you allowed to access or perform?”

5. Account Settings

The final part of Module 1 is account settings.

Users should have a way to manage relevant account-related information. Depending on your design, this can include profile updates, account preferences, and other appropriate account-management functionality.

Make sure that sensitive account operations are protected and that users can modify only their own account information unless the user's role specifically provides administrative access.

6. Backend Implementation

The project specifies **Python and FastAPI** for the backend and **PostgreSQL** as the primary database. MongoDB is listed as a secondary database and Redis is included as the caching technology.

For this module, focus particularly on designing the user-related database structure and backend APIs.

You should think about entities such as:

User → Role → Profile → Authentication Details

The exact database design is part of your implementation and should be properly documented.

The project also lists **SQLAlchemy, Pydantic, and Alembic** among the backend technologies, which can be considered while implementing database models, validation, and database migrations.

7. Frontend Implementation

The project uses **React.js** for the frontend.

For Module 1, the frontend should provide the required screens and navigation for the authentication and user-management flow.

At minimum, think about the following screens:

Registration
Login
Profile
Account Settings
Role-based navigation/dashboard access

The frontend should communicate with the FastAPI backend through APIs. Proper validation and meaningful error messages should be included wherever required.

8. API Testing

Once your APIs are implemented, test them using **Postman**, which is included in the project's development and deployment tools.

Do not test only the successful scenario.

You should also test cases such as:

Valid registration
Duplicate registration
Valid login
Invalid login credentials
Accessing a protected endpoint without authentication
Accessing a resource with an incorrect role
Updating your own profile
Attempting to access another user's protected information
Invalid or missing input fields

This will help you understand how authentication and authorization work in a real application.

9. Expected Module 1 Flow

The overall flow you should aim to achieve is:

User Registration → Login → Authentication Token → Role Identification → Role-Based Access → Profile Management → Account Settings

After successful login, the authenticated user's identity and role should be available to the backend so that protected APIs can verify whether the user is permitted to perform a requested operation.

10. What You Should Complete

By the end of Module 1, you should have a functional foundation for the CreatorIQ platform.

Your implementation should cover:

1. User registration
2. User login
3. Authentication
4. JWT/OAuth2-based authentication as applicable
5. Creator profile management
6. Agency profile management
7. Role-based access control
8. Account settings
9. Protected backend APIs
10. React frontend screens
11. Database integration
12. API testing using Postman

The project milestone specifically expects **project initialization and architecture setup, authentication and dashboard management, functional analytics visualization components, and completion of system design and UI planning by the end of Milestone 1**.

11. Important Points

While implementing this module, do not focus only on making the UI work. You should understand the complete flow between the frontend, backend, database, authentication mechanism, and authorization layer.

Please maintain clean project structure, meaningful variable and API names, proper error handling, input validation, and clear separation between frontend and backend responsibilities.

Also document the APIs and database design you create so that the next modules can build on top of Module 1 without major restructuring.

The main goal of this module is to establish a **secure and reusable user-management foundation** for the entire CreatorIQ platform. Once this foundation is working properly, the upcoming modules such as Content Analytics, Audience Analytics, Growth & Trend Analysis, Revenue Analytics, and Social Media Integration can build upon it.

Please complete the module according to the project requirements and be prepared to explain your implementation, database design, authentication flow, authorization logic, and API testing during the review.

Regards,
Sirisha